Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251214**
Nama Lengkap	**Raffiel Gideon Nyolo Nyolo**
Minggu ke / Materi	**05 / Modular Programming**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Fungsi, Argument dan Parameter

Perhatikan program berikut:

```
nama = input("Masukkan nama: ")
print("Hallo ", nama, " selamat pagi!")
```

Pada program tersebut ada dua fungsi yang digunakan, yaitu **input()** dan **print()**. Kedua program tersebut merupakan fungsi dari Python yang biasanya disebut dengan *built-in function.* Setiap fungsi memiliki kegunaannya masing-masing, input() digunakan untuk membaca input, sedangkan print() digunakan untuk mengeluarkan tulisan di layar.

Secara umum fungsi adalah kumpulan perintah-perintah yang dijadikan satu, memiliki suatu tujuan dan kegunaan khusus serta dapat digunakan ulang. Dalam membuat program yang membutuhkan langkah yang banyak, di butuhkan pengelompokan untuk beberapa kode program menjadi bagian-bagian (block) dari suatu program yang besar. Itu mengapa disebut sebagai modular, karena program terdiri dari beberapa bagian modular yang memiliki kegunaan khusus dan dapat digunakan ulang.

Fungsi dibagi menjadi dua, berdasarkan fungsinya masing-masing, yaitu fungsi bawaan (*build-in-function*) dan fungsi yang dapat dibuat sendiri oleh programmer.

Perhatikan fungsi tambah() berikut ini yang dapat digunakan untuk menghitung jumlah dari dua bilangan yang diberikan:

```
def tambah(a, b):
    hasil = a + b
    return hasil
```

Fungsi tambah tersebut terdiri dari beberapa hal yang perlu diperhatikan:

- "**def**" digunakan untuk mendefinisikan sebuah fungsi.
- Nama fungsi yang dibuat adalah **tambah()**.
- Isi dari fungsi harus anda tuliskan menjorok ke dalam 1 tab. Perhatikan bagian tambah(a,b): sebagai penanda blok.
- Fungsi tambah() membutuhkan dua argument, yang nantinya akan dikenali sebagai parameter **a** dan **b**.
- Fungsi tersebut akan menghasilkan hasil penjumlahan yang dapat ditampung di sebuah variabel. "**return**" digunakan untuk mengembalikan/mengeluarkan nilai dari suatu fungsi.

Penggunaan fungsi tersebut dapat dilihat pada kode program berikut ini:

```
def tambah(a, b):
    hasil = a + b
    return hasil
c = tambah(10, 5)
print(c)
```

Jalannya program tersebut adalah sebagai berikut:

- Program akan lompat ke baris 1 karena fungsi tambah() dipanggil. Pada baris 1 terdapat parameter a dan b. Karena fungsi tambah() dipanggil dengan arguments 10 dan 5, maka parameter a = 10 dan b = 5 (sesuai urutan).
- Program lanjut ke baris 2. Pada baris ini variabel hasil akan berisi 10 + 5 = 15.
- Program lanjut ke baris 3. Ada penggunaan keyword return, diikuti oleh variabel hasil yang nilainya 15. Return di sini menandakan bahwa fungsi tambah() sudah berakhir dan menghasilkan nilai 15 (sesuai dengan variabel hasil).

- Dari baris 3, program akan kembali ke baris 4. Ingat, jika fungsi sudah selesai, program akan kembali ke baris di mana fungsi tersebut dipanggil. Pada baris 4 sekarang nilai c diisi oleh 15 (hasil dari menjalankan fungsi tambah().
- Pada baris 5 tampilkan nilai dari variabel c ke layar. Output yang dihasilkan adalah 15

```
>>> def tambah (a,b):
...     hasil = a + b
...     return hasil
... c = tambah (10, 5)
... print (c)
...
15
```

Gambar 1.1: Hasil dari pemanggilan fungsi tambah()

Return Value

Berdasarkan hasil yang dikeluarkan oleh fungsi, secara umum ada dua jenis yaitu fungsi yang tidak mengembalikan nilai dan fungsi yang mengembalikan nilai. Fungsi yang tidak mengembalikan nilai sering disebut sebagai **void function**. Sebagai contoh, fungsi print_twice() berikut ini adalah fungsi yang tidak mengembalikan nilai:

```
def print_twice(message):
    print(message)
    print(message)
print_twice("Hello World!")
```

```
>>> def print_twice(messege):
...     print (messege)
...     print (messege)
... print_twice ("Hello World")
...
Hello World
Hello World
>>>
```

Gambar 1.2: Hasil dari pemanggilan fungsi print_twice()

Fungsi print_twice() tidak menghasilkan suatu nilai yang dapat digunakan untuk proses berikutnya. Fungsi print_twice() sebenarnya mengembalikan nilai **None** jika anda coba panggil dengan cara berikut:

```
def print_twice(message):
    print(message)
    print(message)
print(print_twice("Hello World"))
```

Output yang dihasilkan adalah:

```
Hello World!
Hello World!
None
```

Berbeda halnya dengan fungsi tambah() berikut:

```
def tambah(a, b, c):
hasil = a + b + c
return hasil
```

Di dalam fungsi tambah() didefinisikan suatu variabel hasil yang diisi oleh a+b+c. Kemudian variabel hasil tersebut dikeluarkan dari fungsi sebagai hasil dari fungsi tambah() dengan keyword **return**. "**return**" digunakan untuk mengeluarkan nilai hasil dari fungsi dan mengakhiri fungsi.

Fungsi tambah() tersebut bisa digunakan untuk mencari hasil penjumlahan tiga bilangan. Hasil dari penjumlahan tiga bilangan tersebut dapat digunakan untuk proses/langkah berikutnya, seperti contoh program untuk mencari rata-rata dari tiga bilangan berikut ini:

```
def tambah(a, b, c):
    hasil = a + b + c
    return hasil
nilai1 = 70
nilai2 = 85
nilai3 = 55
rata_rata = tambah(nilai1, nilai2, nilai3)/3
print(rata_rata)
```

Optional Argument dan Named Argument

Fungsi dapat memiliki optional parameter, yaitu parameter yang bersifat opsional dan memiliki nilai bawaan (default) yang sudah didefinisikan sebelumnya. Untuk mendefinisikan optional parameter, anda harus mendefinisikan nilai bawaannya terlebih dahulu seperti pada contoh berikut ini:

```
def hitung_belanja(belanja, diskon=0):
    bayar = belanja - (belanja * diskon)/100
    return bayar
```

Fungsi hitung_belanja() memiliki dua parameter yaitu **belanja** dan **diskon**. Parameter diskon secara default memiliki nilai 0 (yang artinya 0%). Untuk memanggil fungsi hitung_belanja() tersebut, anda dapat melakukan dengan beberapa cara seperti berikut ini:

```
def hitung_belanja(belanja, diskon=0):
    bayar = belanja - (belanja * diskon)/100
    return bayar
print(hitung_belanja(100000))
print(hitung_belanja(100000, 10))
print(hitung_belanja(100000, 50))
```

Setiap parameter pada fungsi memiliki nama. Karena itu pada pemanggilan fungsi juga dapat disertakan nama parameternya dan tidak perlu sesuai dengan urutan yang diberikan. Perhatikan contoh berikut ini:

```
def cetak(a, b, c):
    print("Nilai a: ",a)
    print("Nilai b: ",b)
    print("Nilai c: ",c)
cetak(20, 30, 40)
```

Anda juga dapat memanggil fungsi cetak() tersebut dengan cara berikut ini:

```
def cetak(a, b, c):
    print("Nilai a: ",a)
```

```
    print("Nilai b: ",b)
    print("Nilai c: ",c)
cetak(a=20,b=30,c=40)
```

Anonymous Function (Lambda)

Anonymous function pada Python adalah fitur tambahan, bukan merupakan fitur utama. Berbeda dengan bahasa-bahasa pemrograman seperti Haskell, Lisp dan Erlang yang merupakan bahasa pemrograman fungsional. Pada Python, digunakan **lambda** untuk mendefinisikan anonymous function. Sebagai contoh, perhatikan fungsi tambah() berikut ini:

```
def tambah(a, b):
    hasil = a + b
    return hasil
print (tambah (10,20))
```

fungsi tambah menggunakan lambda adalah sebagai berikut:

```
tambah = lambda a, b: a + b
print(tambah(10,20))
```

Setiap anonymous function pada Python terdiri dari beberapa bagian berikut ini:

- Keyword: lambda
- Bound variable: argument pada lambda function
- Body: bagian utama lambda, berisi ekspresi atau statement yang menghasilkan suatu nilai.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github: https://github.com/fieliel-star/71251214_raffiel.git

SOAL 1

```python
def cek_angka(a,b,c):
    if (a != b and b != c and a != c) and (a + b == c or b + c == a or a + c == b):
        return True
    else:
        return False

a = int(input("Masukkan angka pertama: "))
b = int(input("Masukkan angka kedua: "))
c = int(input("Masukkan angka ketiga: "))

print(cek_angka(a,b,c))
```

Penjelasan:

Program tersebut merupakan program Python dan berfungsi untuk memeriksa hubungan antara tiga angka yang dimasukkan oleh pengguna. Program dimulai dengan mendefinisikan sebuah fungsi bernama cek_angka(a, b, c) menggunakan kata kunci def. Fungsi ini menerima tiga parameter yaitu a, b, dan c. Di dalam fungsi terdapat pernyataan if yang berisi kondisi logika. Bagian pertama dari kondisi tersebut adalah (a != b and b != c and a != c). Operator != merupakan operator yang berarti “tidak sama dengan”, sehingga digunakan untuk memastikan bahwa ketiga angka yang dimasukkan memiliki nilai yang berbeda. Kondisi ini digabungkan menggunakan operator logika and, yang berarti seluruh kondisi harus bernilai benar agar bagian ini terpenuhi.

Selanjutnya program memeriksa kondisi kedua yaitu (a + b == c or b + c == a or a + c == b). Pada bagian ini digunakan operator aritmatika + untuk menjumlahkan dua angka, kemudian hasilnya dibandingkan dengan angka lainnya menggunakan operator == yang berarti “sama dengan”. Operator logika or merupakan operatro yang cukup salah satu kondisi bernilai benar agar keseluruhan bagian ini terpenuhi. Jika kedua kelompok kondisi tersebut benar, maka fungsi akan mengembalikan nilai True, sedangkan jika tidak terpenuhi maka fungsi akan

mengembalikan False. Setelah fungsi dibuat, program meminta pengguna memasukkan tiga angka menggunakan input() yang kemudian diubah menjadi bilangan bulat dengan int(). Nilai tersebut disimpan ke dalam variabel a, b, dan c, lalu fungsi cek_angka(a, b, c) dipanggil dan hasilnya ditampilkan menggunakan print().

Output:

```
Masukkan angka pertama: 2
Masukkan angka kedua: 3
Masukkan angka ketiga: 5
True
PS D:\Semester_2\Praktikum
Masukkan angka pertama: 1
Masukkan angka kedua: 2
Masukkan angka ketiga: 4
False
```

SOAL 2

```python
def cek_digit_belakang(a,b,c):
    if a % 10 == b % 10 or b % 10 == c % 10 or a % 10 == c % 10:
        return True
    else:
        return False

a = int(input("Masukkan angka pertama: "))
b = int(input("Masukkan angka kedua: "))
c = int(input("Masukkan angka ketiga: "))

print(cek_digit_belakang(a,b,c))
```

Program tersebut mendefinisikan sebuah fungsi bernama cek_digit_belakang(a, b, c) yang bertujuan untuk memeriksa apakah tiga angka memiliki digit terakhir yang sama. Pemeriksaan digit terakhir dilakukan menggunakan operator modulus %. Operator % digunakan untuk mendapatkan sisa hasil bagi dari suatu pembagian. Dalam konteks ini % 10 digunakan untuk mengambil digit terakhir dari setiap angka karena ketika sebuah bilangan dibagi 10, sisanya adalah digit paling belakang dari bilangan tersebut. Setelah itu, program membandingkan digit

terakhir dari setiap angka menggunakan operator perbandingan == yang berfungsi untuk mengecek apakah dua nilai memiliki nilai yang sama.

Pada baris kondisi if, terdapat operator logika or yang akan menghasilkan nilai True jika salah satu dari kondisi yang diperiksa bernilai benar. Dalam program ini, kondisi yang diperiksa adalah apakah digit terakhir a sama dengan b, atau b sama dengan c, atau a sama dengan c. Jika salah satu perbandingan tersebut bernilai benar, maka fungsi akan mengembalikan nilai True melalui perintah return True. Sebaliknya, jika tidak ada digit terakhir yang sama, bagian else akan dijalankan dan fungsi mengembalikan False. Setelah fungsi dibuat, program meminta pengguna memasukkan tiga angka menggunakan input(), lalu mengubahnya menjadi bilangan bulat dengan int(). Terakhir, fungsi dipanggil dengan nilai yang dimasukkan pengguna dan hasilnya ditampilkan menggunakan print().

Output:

```
PS D:\S> python -u "d:\Semes
Masukkan angka pertama: 30
Masukkan angka kedua: 20
Masukkan angka ketiga: 18
True
PS D:\Semester_2\Praktikum A
Masukkan angka pertama: 145
Masukkan angka kedua: 5
Masukkan angka ketiga: 100
True
```

```
PS D:\Semester_2\Praktikum Al
Masukkan angka pertama: 53
Masukkan angka kedua: 8900
Masukkan angka ketiga: 658
False
PS D:\Semester_2\Praktikum Al
Masukkan angka pertama: 1024
Masukkan angka kedua: 14
Masukkan angka ketiga: 94
True
```

```
PS D:\Semester_2\Praktikum
Masukkan angka pertama: 71
Masukkan angka kedua: 187
Masukkan angka ketiga: 18
False
```

SOAL 3

```python
def konversi_suhu(celcius):
    c_ke_f = lambda c: (9/5) * c + 32
    c_ke_r = lambda c: 0.8 * c

    fahrenheit = c_ke_f(celcius)
    reamur = c_ke_r(celcius)
    return fahrenheit, reamur

celcius = float(input("Masukkan suhu dalam Celcius: "))
f, r = konversi_suhu(celcius)

print("Suhu dalam Fahrenheit:", int(f))
print("Suhu dalam Reamur:", int(r))
```

Penjelasan:

Program tersebut digunakan untuk mengonversi suhu dari Celcius ke dua satuan Fahrenheit dan Reamur. Pada awal program dibuat sebuah fungsi bernama konversi_suhu(celcius) yang menerima satu parameter berupa nilai suhu dalam Celcius. Di dalam fungsi tersebut terdapat dua fungsi kecil yang dibuat menggunakan lambda, yaitu c_ke_f dan c_ke_r. Fungsi lambda merupakan cara singkat untuk membuat fungsi sederhana dalam satu baris. Pada c_ke_f = lambda c: (9/5) * c + 32, digunakan operator pembagian (/), perkalian (*), dan penjumlahan (+) untuk mendapatkan nilai suhu dalam Fahrenheit. Sedangkan pada c_ke_r = lambda c: 0.8 * c, hanya digunakan operator perkalian * untuk mengalikan nilai Celcius dengan 0.8 sesuai rumus konversi ke Reamur.

Setelah fungsi konversi didefinisikan, program meminta pengguna memasukkan nilai suhu dalam Celcius menggunakan input(), kemudian nilai tersebut diubah menjadi tipe data pecahan dengan fungsi float() agar bisa perhitungan angka desimal. Nilai tersebut kemudian dikirim ke fungsi konversi_suhu(celcius) dan hasilnya disimpan dalam dua variabel, yaitu f untuk Fahrenheit dan r untuk Reamur. Hal ini dimungkinkan karena fungsi return fahrenheit, reamur mengembalikan dua nilai sekaligus. Pada bagian akhir program, hasil konversi ditampilkan menggunakan print(). Sebelum ditampilkan, nilai f dan r diubah menjadi bilangan bulat (int()),

supaya angka desimalnya hilang. Dengan demikian, memanfaatkan fungsi lambda untuk membuat perhitungan menjadi lebih ringkas.

Output:

```
Masukkan suhu dalam Celcius: 100
Suhu dalam Fahrenheit: 212
Suhu dalam Reamur: 80
PS D:\Semester_2\Praktikum Algor
Masukkan suhu dalam Celcius: 80
Suhu dalam Fahrenheit: 176
Suhu dalam Reamur: 64
```

```
PS D:\Semester_2\Praktikum Algo
Masukkan suhu dalam Celcius: 0
Suhu dalam Fahrenheit: 32
Suhu dalam Reamur: 0
```